Bayescoverage APP - overview

Leontine Alkema, last updated June 1, 2026

Installation info is given in the readme and shinyapp

```
library(bayescoveragedeploy)
```

Where data are stored:

```
data_folder <- "data_raw" # for survey data
routinedata_dir <- here::here("private/routinedata/")
# see note on working with routine dummy data below
```

Choose an indicator. Naming comes from survey data, options include

- c("anc1trimester", "vmsl", "ideliv", "vdpt", "anc4") , with routine as available
- c("bfexcl0_5", "ancq8" , "cci" , "sba"), no routine data

```
indicator_select <-"anc4" #"cci"#"vdpt"#"anc4"
```

Read in survey data and process

```
dat0 <- haven::read_dta(here::here(data_folder, "ICEH_all.long.dta")) |>
  dplyr::rename(r = r_raw, se = se_raw)
# process for indicator_select
dat <- bayescoveragemodel::process_data(
  dat = dat0,
  regions_dat = bayescoveragemodel::regions_all,
  indicator = indicator_select)
```

```
[1] "For ANC4, we combined anc42 and anc43 and call it anc4."
[1] "Some clusters are NA, these observations are excluded for now."
[1] "Observations with missing clusters"
# A tibble: 6 x 3
  iso    year cluster
  <chr> <dbl> <chr>
1 XKX    2013 <NA>
2 XKX    2019 <NA>
3 PBA    2010 <NA>
4 PPU    2011 <NA>
5 PIL    2006 <NA>
6 PIL    2011 <NA>
[1] "Some SEs are NA, these observations are excluded for now."
[1] "We add record_id_fixed to the data."
[1] "Currently no biases assigned."
[1] "Column possible_outlier not included in data, no pre-defined outliers used"
[1] "Remove data that's not DHS or MICS, number of obs removed:"
[1] 6
```

Fitting for one country, use ISO to select

```
# countries with survey data
# unique(dat$iso)
iso_select <- "RWA"
dat_use <-
  dat |>
  dplyr::filter(iso %in% iso_select)
```

Do we want to add routine data?

```
add_routine <- ifelse(
  indicator_select %in% c("anc1trimester", "vmsl", "ideliv", "vdpt", "anc4"),
  TRUE,
  FALSE)
add_routine
```

```
[1] TRUE
```

If yes, read in routine data.

```r
if (!add_routine){
  routine_dat  <- routine_dat_use <- NULL
} else {
  # if we want to use real data
  # Some processing here, for CAM, this would come from app
  # routine_dat0 <- read_csv(
  #   paste0(routinedata_dir, "/dat_combi_",
  #          indicator_select,  ".csv")) |>
  #   mutate(indicator = indicator_select)
  # # routine_dat has indicator_name (name in routine) and
  # # indicator (indicator name from survey data
  # routine_dat  <-
  #   routine_dat0  |>
  #   filter(worst_notmissing >= 90,
  #          worst_notoutliers >= 90,
  #          worst_abs_diff_level < 0.2,
  #          worst_rr >= 60) |>
  #   #|>
  #   #  filter(!is.na(diff_roc))
  #   # remove countries w/o any nonNA diff_roc
  #   group_by(iso) |>
  #   filter(sum(!is.na(diff_roc)) > 0) |>
  #   ungroup() |>
  #   mutate(sd_routine_roc = 0,
  #          sd_routine = 0) |>
  #   mutate(worst_combi =
  #            worst_rr/100*worst_notmissing/100*worst_notoutliers)

  # routine_dat_use <-
  #   routine_dat |>
  #   filter(iso %in% iso_select)

  # or if we want to use dummy data
  routine_dat_use <- readr::read_csv(here::here("data_raw/routine_toydata.csv")) |>
  # add iso
  dplyr::mutate(iso = iso_select,
  # add indicator name (as per routine data naming)
            indicator_name = dplyr::case_when(
              indicator_select == "anc1trimester" ~ "anc",
              indicator_select == "vmsl" ~ "measles2",
              indicator_select == "ideliv" ~ "instdeliveries",
              indicator_select == "vdpt" ~ "penta3",
```

```
              indicator_select == "anc4" ~ "anc4"
            ),
  # add indicator (from survey data naming)
            indicator = indicator_select)

}
```

```
Rows: 5 Columns: 9
-- Column specification --------------------------------------------------------
Delimiter: ","
dbl (6): year, routine_value, routine_roc, worst_combi, sd_routine_roc, sd_r...
lgl (3): iso, indicator_name, indicator

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
routine_dat_use
```

```
# A tibble: 5 x 9
  iso    year routine_value routine_roc worst_combi sd_routine_roc sd_routine
  <chr> <dbl>         <dbl>       <dbl>       <dbl>          <dbl>      <dbl>
1 RWA    2020          0.5        NA            95              0          0
2 RWA    2021          0.51        0.01         95              0          0
3 RWA    2022          0.52        0.01         95              0          0
4 RWA    2023          0.57        0.05         95              0          0
5 RWA    2024          0.54       -0.03         95              0          0
# i 2 more variables: indicator_name <chr>, indicator <chr>
```

```
fit <- bayescoveragedeploy::fit_local_model(
          survey_df = dat |> dplyr::filter(iso %in% iso_select),
          indicator = indicator_select,
          iso_select  = iso_select,
          routine_df = routine_dat_use
        )
```

```
[1] "We use a global fit, and take selected settings from there."
[1] "settings for the spline_degree and num_knots taken from global run"
[1] "settings for Ptilde_low taken from global run"
[1] "Setting for tstar taken from global run"
[1] "Settings for hierarchical settings taken from global run"
```

```
[1] "For hierarchical terms, we fix things up to the 2nd-lowest level."
[1] "We fix all sigmas of hierarchical models."
[1] "We take the data model setting from global fit."
[1] "We add smoothing"
[1] "We fix smoothing and data model parameters."
[1] "We filter data to be inside estimation period"
[1] "We use national data"
[1] "We only use data for RWA"
[1] "NOT using or producing aggregates"


Joining with `by = join_by(iso)`


[1] "Using precompiled Stan model"

CHECKING DATA AND PREPROCESSING FOR MODEL 'fpem_routine' NOW.

COMPILING MODEL 'fpem_routine' NOW.

STARTING SAMPLER FOR MODEL 'fpem_routine' NOW.

CHECKING DATA AND PREPROCESSING FOR MODEL 'fpem_routine' NOW.

COMPILING MODEL 'fpem_routine' NOW.

STARTING SAMPLER FOR MODEL 'fpem_routine' NOW.

SAMPLING FOR MODEL 'fpem_routine' NOW (CHAIN 1).

CHECKING DATA AND PREPROCESSING FOR MODEL 'fpem_routine' NOW.
Chain 1:
Chain 1: Gradient evaluation took 0.003993 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 39.93 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:

COMPILING MODEL 'fpem_routine' NOW.

STARTING SAMPLER FOR MODEL 'fpem_routine' NOW.
Chain 1: Iteration:   1 / 350 [  0%]  (Warmup)

SAMPLING FOR MODEL 'fpem_routine' NOW (CHAIN 2).
```

```
Chain 2:
Chain 2: Gradient evaluation took 8.3e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.83 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:   1 / 350 [  0%]  (Warmup)

CHECKING DATA AND PREPROCESSING FOR MODEL 'fpem_routine' NOW.

COMPILING MODEL 'fpem_routine' NOW.

STARTING SAMPLER FOR MODEL 'fpem_routine' NOW.

SAMPLING FOR MODEL 'fpem_routine' NOW (CHAIN 3).
Chain 3:
Chain 3: Gradient evaluation took 8.8e-05 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.88 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:   1 / 350 [  0%]  (Warmup)

CHECKING DATA AND PREPROCESSING FOR MODEL 'fpem_routine' NOW.
Chain 3: Iteration:  10 / 350 [  2%]  (Warmup)

COMPILING MODEL 'fpem_routine' NOW.

STARTING SAMPLER FOR MODEL 'fpem_routine' NOW.

SAMPLING FOR MODEL 'fpem_routine' NOW (CHAIN 4).
Chain 4:
Chain 4: Gradient evaluation took 0.000104 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 1.04 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:   1 / 350 [  0%]  (Warmup)
Chain 1: Iteration:  10 / 350 [  2%]  (Warmup)
Chain 2: Iteration:  10 / 350 [  2%]  (Warmup)
Chain 1: Iteration:  20 / 350 [  5%]  (Warmup)
Chain 4: Iteration:  10 / 350 [  2%]  (Warmup)
Chain 3: Iteration:  20 / 350 [  5%]  (Warmup)
```

```
Chain 2: Iteration:  20 / 350 [  5%]  (Warmup)
Chain 1: Iteration:  30 / 350 [  8%]  (Warmup)
Chain 1: Iteration:  40 / 350 [ 11%]  (Warmup)
Chain 3: Iteration:  30 / 350 [  8%]  (Warmup)
Chain 4: Iteration:  20 / 350 [  5%]  (Warmup)
Chain 2: Iteration:  30 / 350 [  8%]  (Warmup)
Chain 3: Iteration:  40 / 350 [ 11%]  (Warmup)
Chain 1: Iteration:  50 / 350 [ 14%]  (Warmup)
Chain 4: Iteration:  30 / 350 [  8%]  (Warmup)
Chain 2: Iteration:  40 / 350 [ 11%]  (Warmup)
Chain 1: Iteration:  60 / 350 [ 17%]  (Warmup)
Chain 4: Iteration:  40 / 350 [ 11%]  (Warmup)
Chain 3: Iteration:  50 / 350 [ 14%]  (Warmup)
Chain 1: Iteration:  70 / 350 [ 20%]  (Warmup)
Chain 2: Iteration:  50 / 350 [ 14%]  (Warmup)
Chain 4: Iteration:  50 / 350 [ 14%]  (Warmup)
Chain 3: Iteration:  60 / 350 [ 17%]  (Warmup)
Chain 2: Iteration:  60 / 350 [ 17%]  (Warmup)
Chain 1: Iteration:  80 / 350 [ 22%]  (Warmup)
Chain 3: Iteration:  70 / 350 [ 20%]  (Warmup)
Chain 4: Iteration:  60 / 350 [ 17%]  (Warmup)
Chain 2: Iteration:  70 / 350 [ 20%]  (Warmup)
Chain 1: Iteration:  90 / 350 [ 25%]  (Warmup)
Chain 2: Iteration:  80 / 350 [ 22%]  (Warmup)
Chain 4: Iteration:  70 / 350 [ 20%]  (Warmup)
Chain 3: Iteration:  80 / 350 [ 22%]  (Warmup)
Chain 1: Iteration: 100 / 350 [ 28%]  (Warmup)
Chain 1: Iteration: 110 / 350 [ 31%]  (Warmup)
Chain 4: Iteration:  80 / 350 [ 22%]  (Warmup)
Chain 1: Iteration: 120 / 350 [ 34%]  (Warmup)
Chain 2: Iteration:  90 / 350 [ 25%]  (Warmup)
Chain 3: Iteration:  90 / 350 [ 25%]  (Warmup)
Chain 1: Iteration: 130 / 350 [ 37%]  (Warmup)
Chain 1: Iteration: 140 / 350 [ 40%]  (Warmup)
Chain 4: Iteration:  90 / 350 [ 25%]  (Warmup)
Chain 2: Iteration: 100 / 350 [ 28%]  (Warmup)
Chain 1: Iteration: 150 / 350 [ 42%]  (Warmup)
Chain 3: Iteration: 100 / 350 [ 28%]  (Warmup)
Chain 1: Iteration: 151 / 350 [ 43%]  (Sampling)
Chain 1: Iteration: 160 / 350 [ 45%]  (Sampling)
Chain 2: Iteration: 110 / 350 [ 31%]  (Warmup)
Chain 1: Iteration: 170 / 350 [ 48%]  (Sampling)
Chain 4: Iteration: 100 / 350 [ 28%]  (Warmup)
```

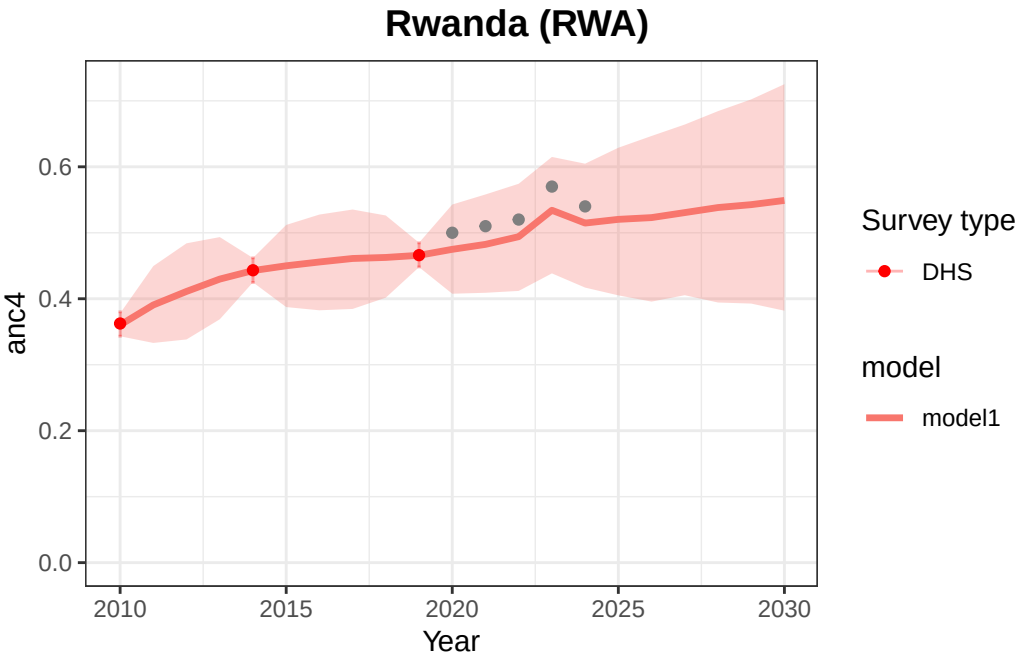
```
Chain 2: Iteration: 120 / 350 [ 34%]  (Warmup)
Chain 3: Iteration: 110 / 350 [ 31%]  (Warmup)
Chain 1: Iteration: 180 / 350 [ 51%]  (Sampling)
Chain 3: Iteration: 120 / 350 [ 34%]  (Warmup)
Chain 4: Iteration: 110 / 350 [ 31%]  (Warmup)
Chain 1: Iteration: 190 / 350 [ 54%]  (Sampling)
Chain 2: Iteration: 130 / 350 [ 37%]  (Warmup)
Chain 3: Iteration: 130 / 350 [ 37%]  (Warmup)
Chain 1: Iteration: 200 / 350 [ 57%]  (Sampling)
Chain 2: Iteration: 140 / 350 [ 40%]  (Warmup)
Chain 3: Iteration: 140 / 350 [ 40%]  (Warmup)
Chain 4: Iteration: 120 / 350 [ 34%]  (Warmup)
Chain 1: Iteration: 210 / 350 [ 60%]  (Sampling)
Chain 3: Iteration: 150 / 350 [ 42%]  (Warmup)
Chain 2: Iteration: 150 / 350 [ 42%]  (Warmup)
Chain 3: Iteration: 151 / 350 [ 43%]  (Sampling)
Chain 2: Iteration: 151 / 350 [ 43%]  (Sampling)
Chain 1: Iteration: 220 / 350 [ 62%]  (Sampling)
Chain 4: Iteration: 130 / 350 [ 37%]  (Warmup)
Chain 3: Iteration: 160 / 350 [ 45%]  (Sampling)
Chain 2: Iteration: 160 / 350 [ 45%]  (Sampling)
Chain 1: Iteration: 230 / 350 [ 65%]  (Sampling)
Chain 4: Iteration: 140 / 350 [ 40%]  (Warmup)
Chain 3: Iteration: 170 / 350 [ 48%]  (Sampling)
Chain 2: Iteration: 170 / 350 [ 48%]  (Sampling)
Chain 1: Iteration: 240 / 350 [ 68%]  (Sampling)
Chain 4: Iteration: 150 / 350 [ 42%]  (Warmup)
Chain 3: Iteration: 180 / 350 [ 51%]  (Sampling)
Chain 4: Iteration: 151 / 350 [ 43%]  (Sampling)
Chain 2: Iteration: 180 / 350 [ 51%]  (Sampling)
Chain 1: Iteration: 250 / 350 [ 71%]  (Sampling)
Chain 4: Iteration: 160 / 350 [ 45%]  (Sampling)
Chain 3: Iteration: 190 / 350 [ 54%]  (Sampling)
Chain 2: Iteration: 190 / 350 [ 54%]  (Sampling)
Chain 1: Iteration: 260 / 350 [ 74%]  (Sampling)
Chain 4: Iteration: 170 / 350 [ 48%]  (Sampling)
Chain 3: Iteration: 200 / 350 [ 57%]  (Sampling)
Chain 2: Iteration: 200 / 350 [ 57%]  (Sampling)
Chain 1: Iteration: 270 / 350 [ 77%]  (Sampling)
Chain 4: Iteration: 180 / 350 [ 51%]  (Sampling)
Chain 3: Iteration: 210 / 350 [ 60%]  (Sampling)
Chain 2: Iteration: 210 / 350 [ 60%]  (Sampling)
Chain 1: Iteration: 280 / 350 [ 80%]  (Sampling)
```

```
Chain 4: Iteration: 190 / 350 [ 54%]  (Sampling)
Chain 3: Iteration: 220 / 350 [ 62%]  (Sampling)
Chain 2: Iteration: 220 / 350 [ 62%]  (Sampling)
Chain 1: Iteration: 290 / 350 [ 82%]  (Sampling)
Chain 3: Iteration: 230 / 350 [ 65%]  (Sampling)
Chain 4: Iteration: 200 / 350 [ 57%]  (Sampling)
Chain 2: Iteration: 230 / 350 [ 65%]  (Sampling)
Chain 1: Iteration: 300 / 350 [ 85%]  (Sampling)
Chain 3: Iteration: 240 / 350 [ 68%]  (Sampling)
Chain 4: Iteration: 210 / 350 [ 60%]  (Sampling)
Chain 2: Iteration: 240 / 350 [ 68%]  (Sampling)
Chain 1: Iteration: 310 / 350 [ 88%]  (Sampling)
Chain 3: Iteration: 250 / 350 [ 71%]  (Sampling)
Chain 4: Iteration: 220 / 350 [ 62%]  (Sampling)
Chain 2: Iteration: 250 / 350 [ 71%]  (Sampling)
Chain 1: Iteration: 320 / 350 [ 91%]  (Sampling)
Chain 3: Iteration: 260 / 350 [ 74%]  (Sampling)
Chain 4: Iteration: 230 / 350 [ 65%]  (Sampling)
Chain 2: Iteration: 260 / 350 [ 74%]  (Sampling)
Chain 1: Iteration: 330 / 350 [ 94%]  (Sampling)
Chain 3: Iteration: 270 / 350 [ 77%]  (Sampling)
Chain 4: Iteration: 240 / 350 [ 68%]  (Sampling)
Chain 2: Iteration: 270 / 350 [ 77%]  (Sampling)
Chain 1: Iteration: 340 / 350 [ 97%]  (Sampling)
Chain 3: Iteration: 280 / 350 [ 80%]  (Sampling)
Chain 4: Iteration: 250 / 350 [ 71%]  (Sampling)
Chain 2: Iteration: 280 / 350 [ 80%]  (Sampling)
Chain 1: Iteration: 350 / 350 [100%]  (Sampling)
Chain 3: Iteration: 290 / 350 [ 82%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.548 seconds (Warm-up)
Chain 1:                0.274 seconds (Sampling)
Chain 1:                0.822 seconds (Total)
Chain 1:
Chain 4: Iteration: 260 / 350 [ 74%]  (Sampling)
Chain 2: Iteration: 290 / 350 [ 82%]  (Sampling)
Chain 3: Iteration: 300 / 350 [ 85%]  (Sampling)
Chain 4: Iteration: 270 / 350 [ 77%]  (Sampling)
Chain 2: Iteration: 300 / 350 [ 85%]  (Sampling)
Chain 3: Iteration: 310 / 350 [ 88%]  (Sampling)
Chain 4: Iteration: 280 / 350 [ 80%]  (Sampling)
Chain 2: Iteration: 310 / 350 [ 88%]  (Sampling)
Chain 3: Iteration: 320 / 350 [ 91%]  (Sampling)
```

```
Chain 4: Iteration: 290 / 350 [ 82%]  (Sampling)
Chain 2: Iteration: 320 / 350 [ 91%]  (Sampling)
Chain 3: Iteration: 330 / 350 [ 94%]  (Sampling)
Chain 4: Iteration: 300 / 350 [ 85%]  (Sampling)
Chain 2: Iteration: 330 / 350 [ 94%]  (Sampling)
Chain 3: Iteration: 340 / 350 [ 97%]  (Sampling)
Chain 4: Iteration: 310 / 350 [ 88%]  (Sampling)
Chain 2: Iteration: 340 / 350 [ 97%]  (Sampling)
Chain 3: Iteration: 350 / 350 [100%]  (Sampling)
Chain 4: Iteration: 320 / 350 [ 91%]  (Sampling)
Chain 3:
Chain 3:  Elapsed Time: 0.574 seconds (Warm-up)
Chain 3:                0.281 seconds (Sampling)
Chain 3:                0.855 seconds (Total)
Chain 3:
Chain 2: Iteration: 350 / 350 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.63 seconds (Warm-up)
Chain 2:                0.29 seconds (Sampling)
Chain 2:                0.92 seconds (Total)
Chain 2:
Chain 4: Iteration: 330 / 350 [ 94%]  (Sampling)
Chain 4: Iteration: 340 / 350 [ 97%]  (Sampling)
Chain 4: Iteration: 350 / 350 [100%]  (Sampling)
Chain 4:
Chain 4:  Elapsed Time: 0.578 seconds (Warm-up)
Chain 4:                0.292 seconds (Sampling)
Chain 4:                0.87 seconds (Total)
Chain 4:

Warning: Bulk Effective Samples Size (ESS) is too low, indicating posterior means and medians
Running the chains for more iterations may help. See
https://mc-stan.org/misc/warnings.html#bulk-ess

Warning: Tail Effective Samples Size (ESS) is too low, indicating posterior variances and tai
Running the chains for more iterations may help. See
https://mc-stan.org/misc/warnings.html#tail-ess

Extracting posteriors...
```

Overview plot

```
countryplot <- bayescoveragemodel::plot_estimates_local_all(
  results = fit,
  save_plots = FALSE)[[1]]
countryplot
```



Estimates

```
fit$posteriors$temporal |>
  dplyr::select(iso, year, `50%`, `2.5%`, `97.5%`)
```

```
# A tibble: 21 x 5
   iso    year `50%` `2.5%` `97.5%`
   <chr> <int> <dbl>  <dbl>   <dbl>
 1 RWA    2010 0.360  0.343   0.378
 2 RWA    2011 0.391  0.333   0.450
 3 RWA    2012 0.411  0.338   0.484
 4 RWA    2013 0.430  0.369   0.493
 5 RWA    2014 0.443  0.425   0.462
 6 RWA    2015 0.450  0.387   0.512
 7 RWA    2016 0.456  0.382   0.528
 8 RWA    2017 0.461  0.385   0.535
 9 RWA    2018 0.463  0.402   0.526
10 RWA    2019 0.466  0.448   0.485
```

```
# i 11 more rows
```